23.2 -Capstone Project Workshop: End-to-End Application Development

Name:MOHAMMAD JAFFER SHAREEF

Hno:2303A51496

Smart Student Attendance & Performance Tracker:

Task-1 :Project proposal

Objective:

To develop a web-based application that helps colleges track student attendance, monitor performance, and generate reports using AI-assisted tools.

Features

- Student login & admin login
- Mark attendance (daily/subject-wise)
- View attendance percentage
- Alerts for low attendance
- Performance tracking (marks)
- Dashboard with charts
- Export reports (PDF/Excel)

Tools & Technologies:

- **Frontend:** HTML, CSS, JavaScript (or React)
- **Backend:** Python (Flask) / Node.js
- **Database:** MySQL / SQLite **AI Tools:** ChatGPT, GitHub Copilot **Deployment:**
  - GitHub Pages (frontend)
    Render / Heroku (backend)

Expected Output

- A working web application
- Dashboard showing attendance stats
- Admin can manage students
- Deployed live project link

Task 2 – Application Design

Architecture (Simple)

Frontend (HTML/CSS/JS)
↓
Backend API (Flask / Node.js)
↓
Database (MySQL)

Database Tables

- Students (id, name, dept)
- Attendance (id, student_id, date, status)
- Marks (id, student_id, subject, score)

UI Pages

- Login Page
- Dashboard
- Attendance Page
- Reports Page

Task 3 – Development Plan

Phase 1 – Frontend

- Create forms (login, attendance)
- Dashboard UI

Phase 2 – Backend

- API:
  - /login
  - /mark-attendance
  - /get-attendance
  - /get-report

Phase 3 – Integration

- Connect frontend with backend using fetch/AJAX **Task 4 –**

Documentation

Include in README.md:

- Project description
- Setup steps
- Technologies used

- Screenshots **Task 5 – Testing**

Use AI to:

- Generate test cases
- Check errors
- Improve performance **Example:**

- Test login validation
- Test attendance calculation **Task 6 – Deployment:**

- Upload code to GitHub Deploy:
  - Frontend → GitHub Pages
    - Backend → Render / Heroku

```
attendance-tracker/
│
├── backend/
│   ├── app.py
│   ├── database.db (auto created)
│
├── frontend/
│   ├── index.html
│   ├── dashboard.html
│   ├── script.js
│   ├── style.css
│
└── README.md
```

Backend (Flask API):

attendance_tracker/backend/app.py

EXPLORER

AI

frontend

dashboard.html

index.html

script.js

app.py

database.db

app.py | index.html | dashboard.html | script.js

app.py > report

```python
from flask import Flask, request, jsonify
import sqlite3
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

# ---------- DATABASE ----------
def get_db():
    conn = sqlite3.connect("database.db")
    conn.row_factory = sqlite3.Row
    return conn

# ---------- HOME ROUTE (IMPORTANT) ----------
@app.route('/')
def home():
    return "Server Running Successfully!"

# ---------- INIT DATABASE ----------
@app.route('/init')
def init():
    conn = get_db()
    cur = conn.cursor()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

EXPLORER

AI

frontend

dashboard.html

index.html

script.js

app.py

database.db

app.py | index.html | dashboard.html | script.js

app.py > report

```python
def init():

    cur.execute("CREATE TABLE IF NOT EXISTS students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, dept T
    cur.execute("CREATE TABLE IF NOT EXISTS attendance(id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTE
    cur.execute("CREATE TABLE IF NOT EXISTS marks(id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER,

    conn.commit()
    conn.close()
    return "Database Ready!"

# ---------- ADD STUDENT ----------
@app.route('/add-student', methods=['POST'])
def add_student():
    data = request.json
    conn = get_db()
    conn.execute("INSERT INTO students(name, dept) VALUES(?,?)",
                 (data['name'], data['dept']))
    conn.commit()
    conn.close()
    return jsonify({"msg": "Student Added"})

# ---------- GET STUDENTS ----------
@app.route('/students')
```

Frontend: attendance_tracker/frontend/index.html (Add Student Page)

app.py | index.html | dashboard.html | script.js

frontend > index.html > html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Student App</title>
</head>
<body>

<h2>Add Student</h2>

<input id="name" placeholder="Name">
<input id="dept" placeholder="Department">
<button onclick="addStudent()">Add</button>

<br><br>
<a href="dashboard.html">Go Dashboard</a>

<script src="script.js"></script>

</body>
</html>
```

attendance_tracker /frontend/dashboard.html

```
D: > AI Assisted Coding > attendance_tracker > frontend > <> dashboard.html > html
<html>
<body>
<h2>Attendance Dashboard</h2>

<button onclick="loadStudents()">Load Students</button>

<ul id="studentList"></ul>

<button onclick="submitAttendance()">Submit Attendance</button>

<h3>Report</h3>
<button onclick="getReport()">Get Report</button>
<ul id="report"></ul>

<script src="script.js"></script>
</body>
</html>
```

attendance_tracker /frontend/script.js

OUTPUT:

Database Ready!

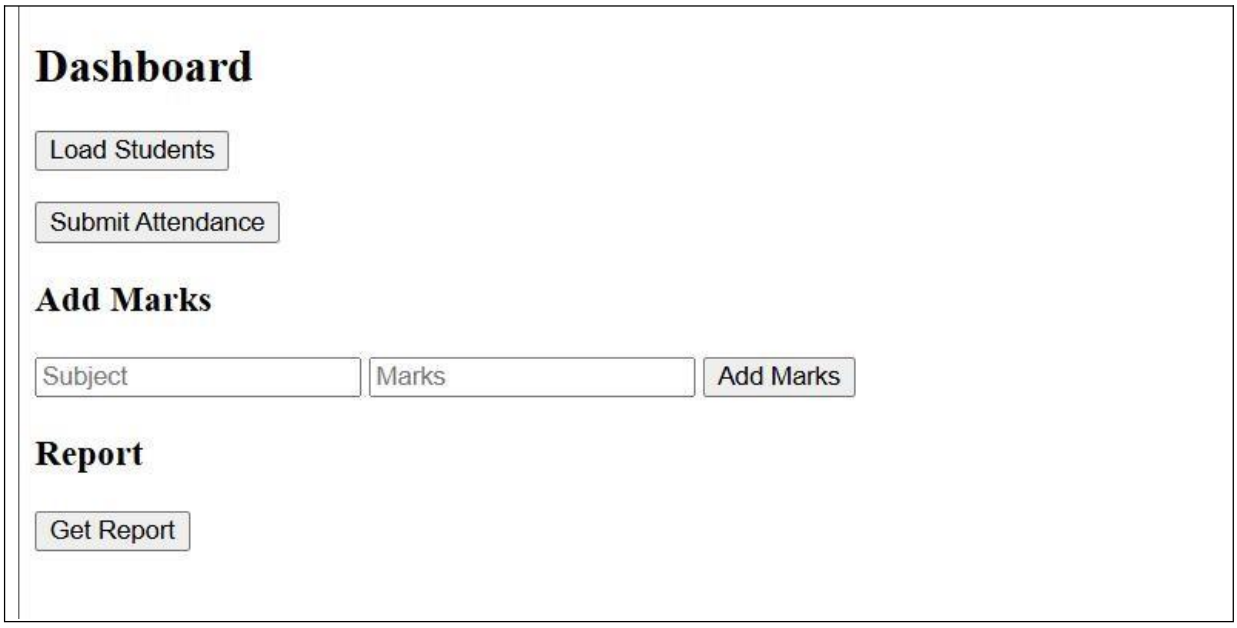
app.py | index.html | dashboard.html | script.js

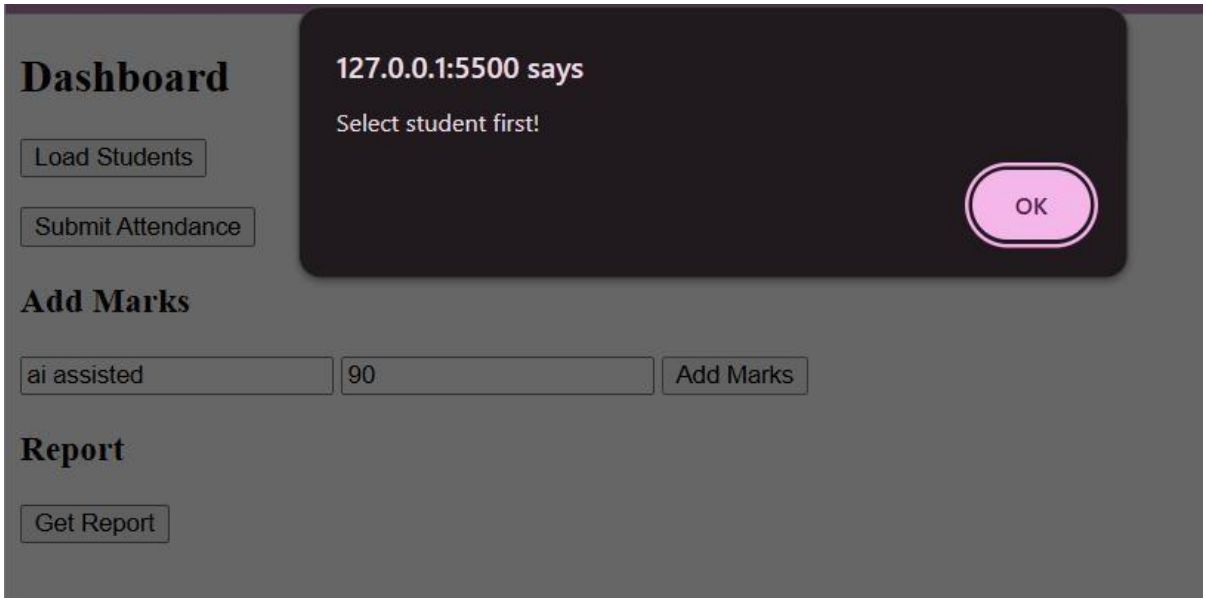
frontend > script.js > ...

```javascript
const API = "http://127.0.0.1:5000";

function addStudent() {
    const name = document.getElementById("name").value;
    const dept = document.getElementById("dept").value;

    fetch(API + "/add-student", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({name, dept})
    })
    .then(res => res.json())
    .then(data => alert(data.msg));
}

function loadStudents() {
    fetch(API + "/students")
    .then(res => res.json())
    .then(data => {
        const list = document.getElementById("list");
        list.innerHTML = "";
```

127.0.0.1:5500/frontend/index.html
Add Student
eshwar
cse
Add
Go Dashboard

127.0.0.1:5500/frontend/index.html
Add Student
eshwar
Go Dashboard
127.0.0.1:5500 says
Student Added
OK


Dashboard
Load Students
Submit Attendance
Add Marks
Subject
Marks
Add Marks
Report
Get Report


Dashboard
Load Students
Submit Attendance
127.0.0.1:5500 says
Select student first!
OK
Add Marks
ai assisted
90
Add Marks
Report
Get Report